EECS 3311

Software Design

Course Project

Service Booking & Consulting Platform

Phase 1 & Phase 2 – Complete Project Specification

York University
Lassonde School of Engineering
School of Information Technology

February 9, 2026

Contents

1 Project Overview **3**

2 Phase 1 – Objective **3**

3 System Description **3**
3.1 Business Context . . . 3
3.2 System Actors . . . 4

4 Use Cases **4**
4.1 Client Use Cases . . . 4
4.1.1 UC1 – Browse Consulting Services . . . 4
4.1.2 UC2 – Request a Booking . . . 4
4.1.3 UC3 – Cancel a Booking . . . 4
4.1.4 UC4 – View Booking History . . . 4
4.1.5 UC5 – Process Payment . . . 4
4.1.6 UC6 – Manage Payment Methods . . . 4
4.1.7 UC7 – View Payment History . . . 4
4.2 Consultant Use Cases . . . 5
4.2.1 UC8 – Manage Availability . . . 5
4.2.2 UC9 – Accept or Reject Booking Request . . . 5
4.2.3 UC10 – Complete a Booking . . . 5
4.3 Admin Use Cases . . . 5
4.3.1 UC11 – Approve Consultant Registration . . . 5
4.3.2 UC12 – Define System Policies . . . 5

5 Booking Lifecycle **5**
5.1 Payment Processing Details . . . 6
5.1.1 Simulated Payment Methods . . . 6
5.1.2 Payment Processing Workflow . . . 6

6 UML Requirements **6**
6.1 Use Case Diagram . . . 6
6.2 Class Diagram (Backend Focus) . . . 6

7 Implementation Requirements **7**
7.1 Backend . . . 7
7.2 Frontend (Phase 1 – Minimal) . . . 7

8 Team Work Expectations **7**

9 Phase 1 Deliverables **8**
9.1 Required Submissions . . . 8
9.2 Submission Format . . . 9

10 Phase 2 – Frontend Completion, Deployment & AI Integration **9**
10.1 Objective . . . 9
10.2 Functional Scope . . . 9

10.2.1 1. Frontend Completion . . . 9
10.2.2 2. Docker-Based Deployment . . . 10
10.2.3 3. AI-Based Customer Assistant . . . 10
10.3 Phase 2 Deliverables . . . 12
10.4 Submission Format . . . 13

11 GitHub Version Control Requirements **14**
11.1 GitHub Repository Setup . . . 14
11.2 What NOT to Include in GitHub . . . 14
11.3 Evaluation of GitHub Usage . . . 14

12 Additional Resources **15**
12.1 Recommended Design Pattern References . . . 15
12.2 UML Modeling Tools . . . 15

13 Important Notes **15**

1 Project Overview

In this project, students will design and implement a **Service Booking & Consulting Platform** that allows clients to book consulting services offered by consultants. The focus of the project is on object-oriented design, clean backend architecture, and correct use of design patterns, supported by UML modeling.

Team Structure: This project is completed in teams of four students.
Phases: The project consists of two phases:
- **Phase 1:** Software Design & Core Implementation
- **Phase 2:** Frontend Completion, Deployment & AI Integration

2 Phase 1 – Objective

The goal of Phase 1 is to design and implement the core backend logic and a minimal frontend while applying software design principles and design patterns. Students must model the system using use case diagrams and class diagrams, and ensure that the implementation aligns with these models.

Key Requirements:
- Create comprehensive UML diagrams (use case and class diagrams)
- Implement core backend business logic
- Apply 3–5 design patterns from the GoF catalog
- Develop a minimal frontend for demonstration
- Ensure consistency between design and implementation

3 System Description

3.1 Business Context

The platform connects **Clients** with **Consultants** who offer professional consulting services (e.g., software consulting, career advising, technical support). The system manages:

- Consulting services
- Consultant availability
- Booking requests
- Booking lifecycle (request, confirmation, cancellation, completion)
- Payment processing (simulated)
- System notifications
- Administrative oversight

Note: The platform does not handle real payments or integrate with actual payment gateways. All payment logic is simulated to demonstrate proper design patterns and workflow implementation.

3.2 System Actors

The system has three primary actors:

1. **Client** – A user who browses services and books consulting sessions.
2. **Consultant** – A service provider who offers consulting sessions and manages availability.
3. **Admin** – A platform administrator who oversees consultants and system policies.

4 Use Cases

Students must create a use case diagram including the following use cases and implement them in the system.

4.1 Client Use Cases

4.1.1 UC1 – Browse Consulting Services

The client views a list of available consulting services, including service type, duration, and base price.

4.1.2 UC2 – Request a Booking

The client selects a consultant and an available time slot and submits a booking request. The system validates availability and creates a booking in the `Requested` state.

4.1.3 UC3 – Cancel a Booking

The client cancels an existing booking. The system applies cancellation rules (e.g., time-based policy) and updates the booking state accordingly.

4.1.4 UC4 – View Booking History

The client views past and upcoming bookings with their current status.

4.1.5 UC5 – Process Payment

The client processes payment for a confirmed booking. The system supports multiple payment methods (credit card, debit card, PayPal, bank transfer). Payment processing is simulated with validation of payment method details and generation of a payment confirmation.

4.1.6 UC6 – Manage Payment Methods

The client can add, view, update, or remove saved payment methods from their account. Payment method information is stored securely (simulated).

4.1.7 UC7 – View Payment History

The client views their complete payment history, including successful payments, pending payments, and refunds.

4.2 Consultant Use Cases

4.2.1 UC8 – Manage Availability

The consultant defines or updates available time slots for consulting sessions.

4.2.2 UC9 – Accept or Reject Booking Request

The consultant reviews incoming booking requests and either accepts or rejects them. Accepting a request confirms the booking; rejecting it notifies the client.

4.2.3 UC10 – Complete a Booking

After a consulting session occurs, the consultant marks the booking as completed.

4.3 Admin Use Cases

4.3.1 UC11 – Approve Consultant Registration

The admin reviews new consultant registrations and approves or rejects them.

4.3.2 UC12 – Define System Policies

The admin configures system-wide policies such as:

- Cancellation rules
- Pricing strategies
- Notification settings
- Refund policies

5 Booking Lifecycle

> Bookings move through well-defined states. This lifecycle must be reflected in both the class design and the implementation.

The booking states include:

State	Description
Requested	Initial state when a client submits a booking request
Confirmed	Consultant has accepted the booking request
Pending Payment	Booking confirmed but awaiting payment
Paid	Payment successfully processed
Rejected	Consultant has declined the booking request
Cancelled	Client or consultant has cancelled the booking
Completed	The consulting session has been completed

> **Payment Flow:** After a consultant confirms a booking, it enters the `Pending Payment` state. Once the client successfully processes payment, it moves to `Paid`. The session can only be marked as `Completed` after payment is processed.

5.1 Payment Processing Details

5.1.1 Simulated Payment Methods

The system must support multiple payment methods, each with different validation requirements:

Payment Method	Simulated Validation
Credit Card	Validate card number format (16 digits), expiry date (future date), CVV (3-4 digits)
Debit Card	Same as credit card validation
PayPal	Validate email format
Bank Transfer	Validate account number format and routing number

5.1.2 Payment Processing Workflow

1. Client selects or adds a payment method
2. System validates payment method details
3. System simulates payment processing (2-3 second delay)
4. System generates a unique transaction ID
5. System updates booking state to `Paid`
6. System sends payment confirmation notification

These policies can be customized by the admin through UC12.

6 UML Requirements

6.1 Use Case Diagram

The use case diagram must include:

- All actors listed in Section 3.2
- All use cases listed in Section 4
- Relationships (`«include»` and `«extend»`) where appropriate
- Clear presentation that is consistent with the implementation

6.2 Class Diagram (Backend Focus)

The class diagram must clearly show:

- **Domain entities** (e.g., `Booking`, `Consultant`, `Service`)

- **Service layer classes**
- **Interfaces and abstract classes**
- **Relationships** (association, inheritance, dependency, aggregation, composition)
- **Pattern-related structures** (clearly showing where design patterns are applied)

> **Important:** The class diagram should demonstrate proper object-oriented design principles including separation of concerns, single responsibility, and appropriate abstraction levels.

7 Implementation Requirements

7.1 Backend

The backend implementation must:

1. Implement the core business logic for all use cases
2. Be consistent with the class diagram
3. Apply **3 to 5 design patterns** from the Gang of Four (GoF) catalog

> **Critical Requirement:** Each design pattern must be:
> - Clearly identifiable in the code
> - Used for a real design problem
> - Not artificially added without purpose

7.2 Frontend (Phase 1 – Minimal)

The frontend should provide a simple UI OR Terminal Command to demonstrate:

- Browsing services
- Booking a session
- Consultant accepting/rejecting bookings
- Processing payment (simulation)

> **Note:** No advanced styling is required. The focus is on functionality and interaction with the backend APIs. Payment processing should demonstrate selection of payment method and confirmation of successful payment.

8 Team Work Expectations

Each team member must:

1. Be responsible for a specific subsystem
    - Examples: booking logic, availability management, policy system, user interface
2. Contribute to UML design (both use case and class diagrams)

3. Contribute to backend or frontend implementation
4. Document their contributions in the README
5. Should know everything in the project and answer to the questions

Version Control (MANDATORY): Teams MUST use GitHub for version control. All team members must commit regularly to the repository. The GitHub repository must include:
- All source code (backend and frontend)
- UML diagrams (source files if possible)
- Documentation (README, etc.)
- Docker configuration files
- Regular commits from all team members showing individual contributions

The GitHub repository URL must be included in your README and submission.

Collaboration: Teams should establish clear communication channels and use version control (e.g., Git) to manage code contributions effectively.

9 Phase 1 Deliverables

9.1 Required Submissions

1. **GitHub Repository URL** (MANDATORY)
   - Repository must contain all project files
   - All team members must have commit history
   - Repository must be accessible to instructors
2. **Use Case Diagram** (PDF or image format)
3. **Class Diagram** (PDF or image format)
4. **Backend Source Code**
   - Well-organized, documented code
   - Clear package/module structure
5. **README Document** explaining:
   - Architecture overview
   - Which design patterns were used and where
   - How to run the application
   - GitHub repository URL
   - Team member contributions (visible from Git commit history)

9.2 Submission Format

MANDATORY GitHub Repository: Your project MUST be hosted on GitHub with regular commits from all team members. Include the repository URL in your README.

All deliverables should be submitted as a single compressed archive (ZIP or TAR) with the following structure:

```
TeamName_Phase1/
|-- diagrams/
|   |-- use_case_diagram.pdf
|   +-- class_diagram.pdf
|-- backend/
|   +-- [source code files]
|-- frontend (if there is)/
|   +-- [source code files]
+-- README.md (MUST include GitHub repository URL)
```

Note: The GitHub repository should contain the same files as your submission archive. We will review commit history to verify individual contributions.

10 Phase 2 – Frontend Completion, Deployment & AI Integration

10.1 Objective

The goal of Phase 2 is to extend the Phase 1 system by completing the frontend, deploying the system using Docker, and integrating an AI-based assistant to support system maintenance and administration.

Building on Phase 1: Phase 2 builds directly on Phase 1. Major architectural changes are discouraged. Students should focus on extension rather than redesign.

10.2 Functional Scope

10.2.1 1. Frontend Completion

Students must enhance the frontend developed in Phase 1 to support all core system workflows, including:

Client Features:

- Browse services
- Request bookings
- Cancel bookings
- View booking history
- Process payments

- Manage payment methods
- View payment history
- Access AI Customer Assistant chatbot

Consultant Features:

- Manage availability
- Accept or reject booking requests
- View booking schedule

Admin Features:

- Approve consultants
- View system status
- Manage system policies

> **UI Design Note:** The frontend does not need advanced UI design; clarity and correctness are the priority.

10.2.2 2. Docker-Based Deployment

The complete system must be deployed using Docker (Minimum 3 Containers).

Minimum Requirements:

- Backend service container
- Frontend service container
- Database container
- `docker-compose.yml` to run the system locally

> **Single Command Deployment:** The system must be runnable with a single command: `docker-compose up`

Docker Configuration Expectations:

- All services should be properly networked
- Database should persist data using volumes
- Environment variables should be configured appropriately
- Port mappings should be clearly documented

10.2.3 3. AI-Based Customer Assistant

The system must include an AI-powered Customer Assistant (Chatbot) accessible to clients.

Purpose The AI assistant helps clients by:

- Answering questions about the consulting platform and how it works
- Providing information about available services and consultants
- Helping clients understand the booking process
- Explaining payment methods and policies
- Assisting with common troubleshooting questions
- Guiding users through the platform features

Requirements

Critical Requirements:

- The AI assistant must be integrated through a backend service
- The chatbot should be accessible from the client interface
- Only general platform information and aggregated data may be provided to the AI
- The AI must not directly access the database
- The AI should not have access to personal user data or private booking details
- AI responses should be informative and helpful, not perform automated actions

Privacy and Safety: The AI chatbot should never have access to personal information, payment details, or confidential booking data. Only provide the AI with general platform information and publicly available service descriptions.

Implementation Guidance The AI Customer Assistant can be implemented using:

- OpenAI API (GPT-4 or GPT-3.5)
- Anthropic Claude API
- Google Gemini API
- Any other LLM service

Example workflow:

1. Client opens the chatbot interface in the frontend
2. Client types a question (e.g., "How do I book a consultant?" or "What payment methods are accepted?")
3. Frontend sends the question to the backend API
4. Backend constructs a prompt with:
    - The user's question
    - System context (platform features, general policies)
    - Available services (public information only)
5. AI service returns a helpful response

6. Backend sends the response back to the frontend
7. Client sees the answer in the chat interface

Example Use Cases for the Chatbot

- **Q:** "How do I book a consulting session?"

  A: AI explains the booking process step-by-step
- **Q:** "What payment methods do you accept?"

  A: AI lists available payment methods (credit card, PayPal, etc.)
- **Q:** "Can I cancel my booking?"

  A: AI explains cancellation policies and refund rules
- **Q:** "What types of consulting services are available?"

  A: AI describes available service categories

> **Evaluation Focus:** The feature should demonstrate proper system integration and design reasoning, not AI model training. You are evaluated on integration quality, not AI performance.

10.3 Phase 2 Deliverables

1. **GitHub Repository (MANDATORY - Updated from Phase 1)**
   - All Phase 2 code and Docker configurations
   - Continued commit history from all team members
   - Clear separation or tagging of Phase 1 vs Phase 2 work
2. **Completed Frontend Source Code**
   - Full implementation of all user workflows
   - Integration with backend APIs
   - Responsive and functional UI
   - AI chatbot interface integrated
3. **Docker Configuration Files**
   - `Dockerfile` for each service
   - `docker-compose.yml` for orchestration
   - `.env.example` showing required environment variables
4. **Running Deployed System**
   - System must start with `docker-compose up`
   - All services must be accessible

- Database must be properly initialized

5. **AI Customer Assistant Documentation**
   - Description of chatbot functionality
   - Examples of questions the chatbot can answer
   - System context provided to the AI
   - Privacy and safety measures implemented
   - API integration approach
6. **Updated README**
   - How to build and run with Docker
   - How to access the AI Customer Assistant chatbot
   - Any API keys or configuration needed
   - GitHub repository URL (same as Phase 1)

10.4 Submission Format

> **MANDATORY GitHub Repository:** Continue using the same GitHub repository from Phase 1. All Phase 2 work must be committed with clear history showing progression from Phase 1.

All deliverables should be submitted as a single compressed archive with the following structure:

```
TeamName_Phase2/
|-- diagrams/
|   |-- use_case_diagram.pdf
|   +-- class_diagram.pdf
|-- backend/
|   |-- src/
|   +-- Dockerfile
|-- frontend/
|   |-- src/
|   +-- Dockerfile
|-- docker-compose.yml
|-- .env.example
|-- AI_CHATBOT_DOCUMENTATION.md
+-- README.md (MUST include GitHub repository URL)
```

> **Note:** The same GitHub repository from Phase 1 should be used. We will review the entire commit history across both phases to verify individual contributions and project evolution.

11 GitHub Version Control Requirements

MANDATORY: All teams must use GitHub for version control throughout both phases of the project.

11.1 GitHub Repository Setup

Repository Structure Your GitHub repository must include:

- All source code (backend and frontend)
- UML diagrams (preferably source files like .plantuml or .drawio)
- Documentation files (README.md, design documents)
- Docker configuration files (Dockerfile, docker-compose.yml)
- `.gitignore` file (exclude dependencies, build artifacts, API keys)
- `.env.example` (template for environment variables, NO actual API keys)

Commit Requirements

- **All team members** must have regular commits throughout the project
- Commit messages must be descriptive (e.g., "Implement payment strategy pattern" not "update")
- Commits should be atomic (one logical change per commit)
- Minimum of 2-3 commits per week per team member

Branching Strategy (Recommended)

- `main` branch for stable, working code
- Feature branches for developing new features
- Pull requests for code review before merging to main

11.2 What NOT to Include in GitHub

Never commit:

- API keys or secrets (use `.env` files locally, provide `.env.example`)
- Dependencies/libraries (use package.json, requirements.txt, etc.)
- Compiled code or build artifacts
- IDE-specific files (.idea/, .vscode/ - add to .gitignore)
- Large binary files or datasets

11.3 Evaluation of GitHub Usage

Your GitHub repository will be evaluated based on:

- **Contribution distribution:** All members must contribute

- **Commit quality:** Descriptive messages, logical commits
- **Commit frequency:** Regular activity throughout the project
- **Repository organization:** Clean structure, proper .gitignore
- **Documentation:** Clear README with setup instructions

> **Tip:** Start using GitHub from Day 1! Don't wait until the end to upload all your code. Regular commits show your work progression and individual contributions.

12 Additional Resources

12.1 Recommended Design Pattern References

- *Design Patterns: Elements of Reusable Object-Oriented Software* by Gamma et al. (Gang of Four)
- *Head First Design Patterns* by Freeman et al.

12.2 UML Modeling Tools

- Lucidchart
- Draw.io (diagrams.net)
- PlantUML
- Visual Paradigm
- StarUML

13 Important Notes

> **Academic Integrity:** All work must be original. Collaboration is expected within teams, but copying code or designs from other teams or external sources is strictly prohibited.

> **Questions?** If you have questions about the project requirements, please:
> - Post on the course discussion forum
> - Attend office hours
> - Email the instructor with specific questions